

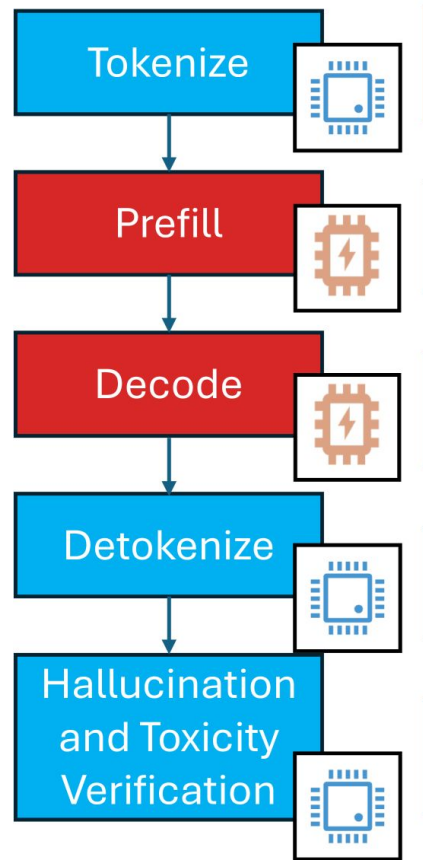
LLM Inference

Что такое инференс?

Инференс — это **процесс выполнения модели** на новых входных данных

По сути: «запуск» обученной модели, чтобы получить ответ

- Для LLM: ввод → токены → генерация → вывод
- Для энкодеров: ввод → токены → вектор/скор



(a)

В чем отличие от обычного использования/обучения?

Отличие от обучения:

- **нет градиентов**, только forward-pass

Обычное использование = запуск в ноутбуке, прототип

Продакшен-инференс требует:

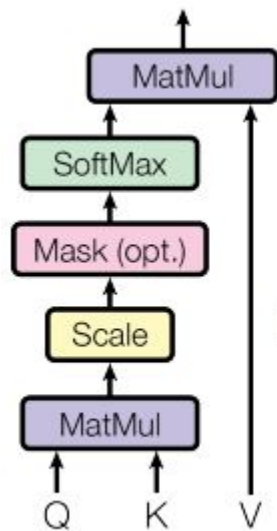
- Быстрого ответа (низкая *latency*)
- Масштабирования (*throughput* и параллельность)
- Предсказуемости и стабильности
- Мониторинга

Cache

KV Cache

KV Cache (Key-Value Cache) — это способ сохранить промежуточные вычисления attention-механизма для уже обработанных токенов

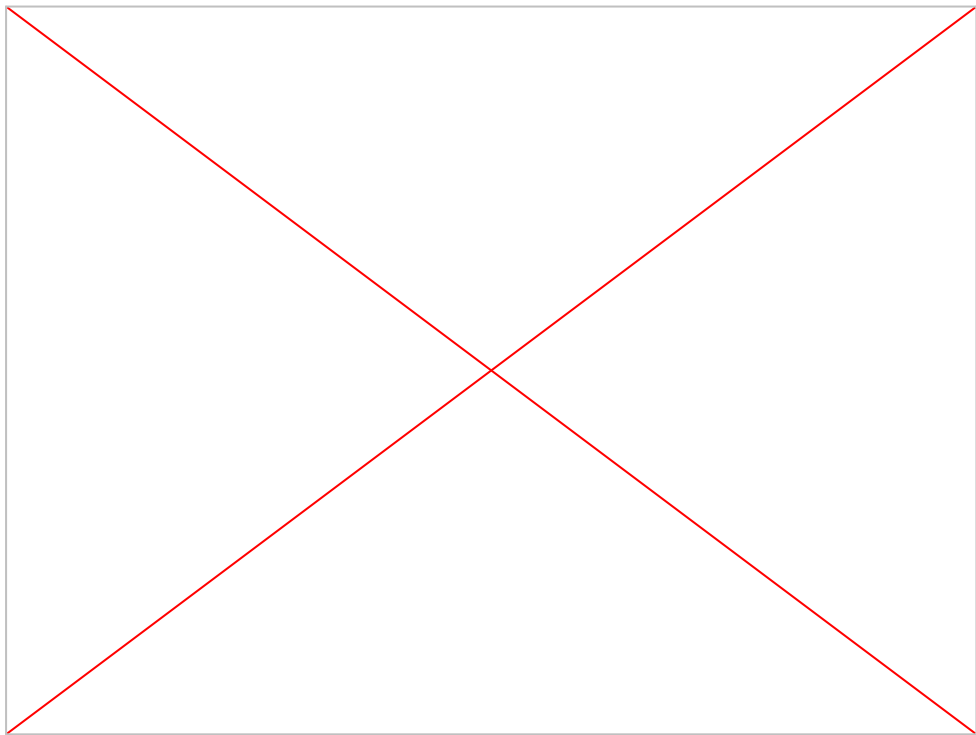
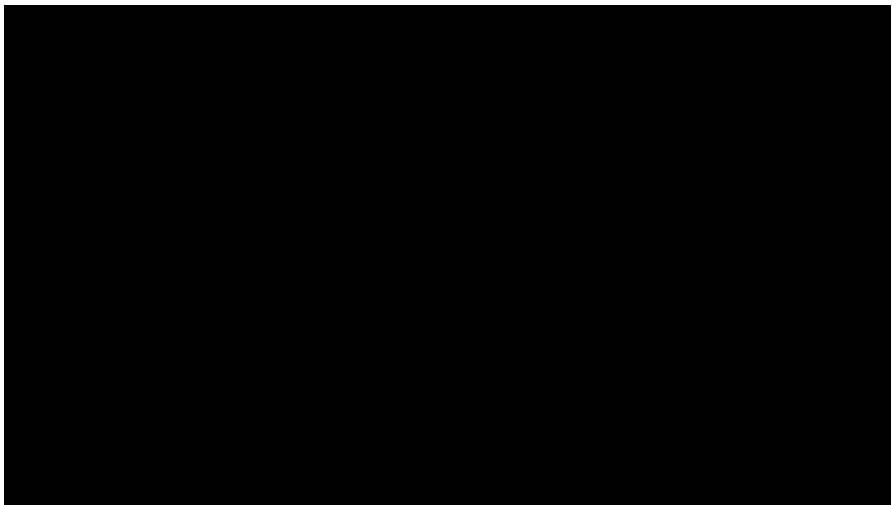
- Используется только при **авторегрессивной генерации** (LLMs)
- Вместо перерасчета attention для всех токенов — переиспользуем сохранённые K и V
- Ускоряет **decode** фазу (каждый новый токен → одна attention-операция с кэшем)



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$



Inference Engine

Scheduler

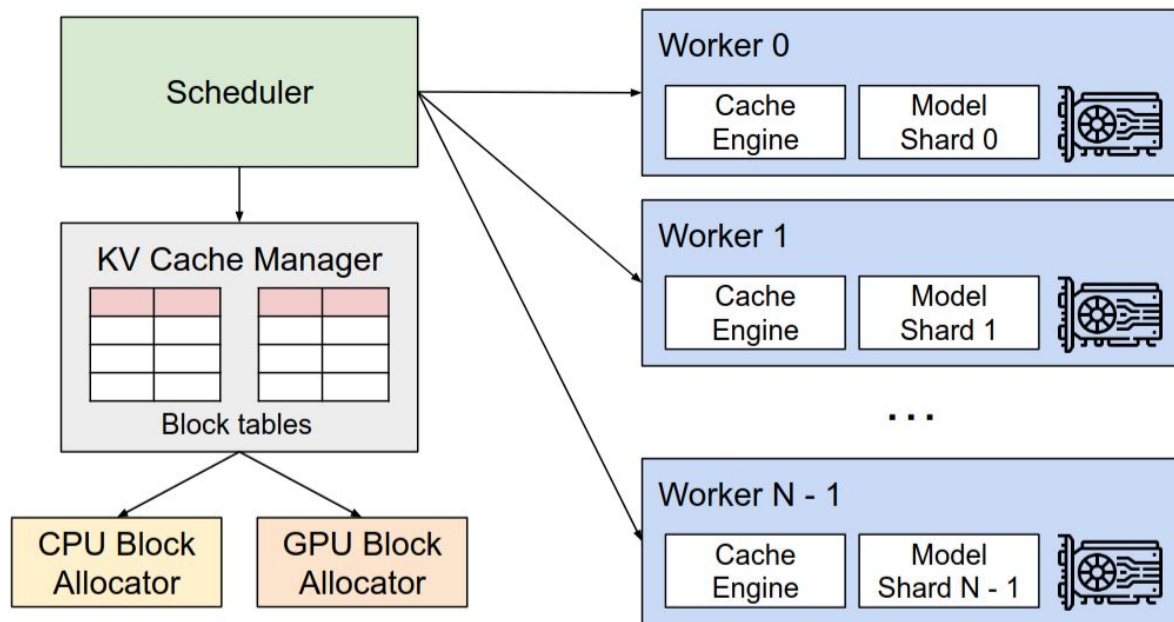


Figure 4. vLLM system overview.

Dynamic Batch

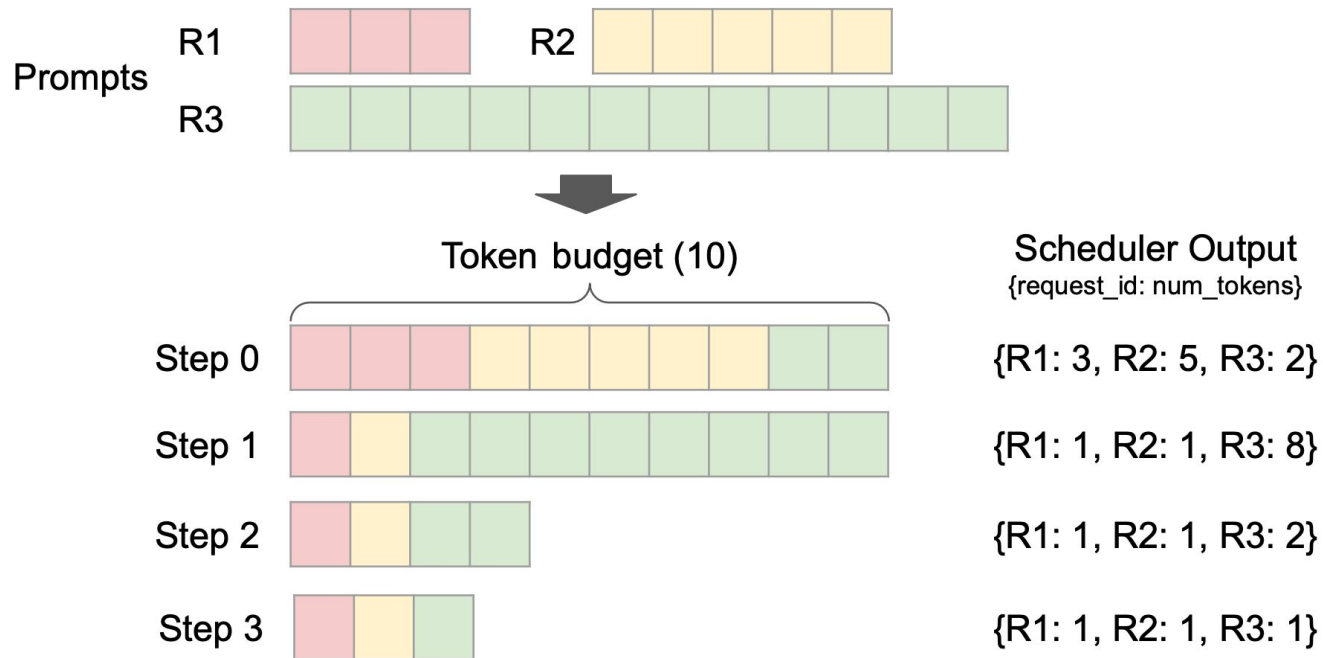
Постоянное объединение новых запросов в уже идущие батчи

Зачем нужно:

- Избегаем простоев GPU
- Увеличиваем throughput
- Снижаем «tail latency» (p95/p99)

Как работает:

- Запросы добавляются динамически
- Слоты в батче перераспределяются по мере завершения
- Нет жёсткой границы «сначала собрать, потом запустить»



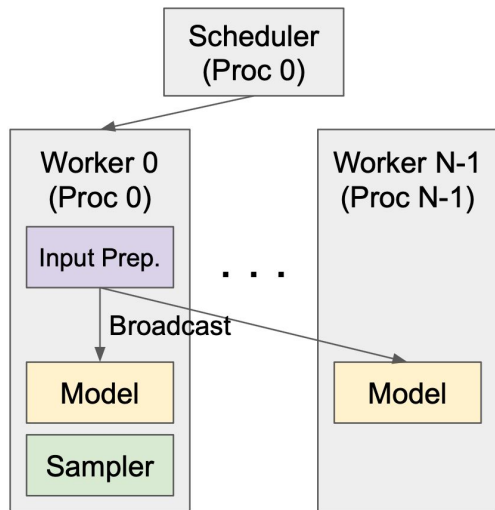
Scheduler

Оркестрация prefill → decode фаз, слотов токенов, позиции в batch, завершения сессий.

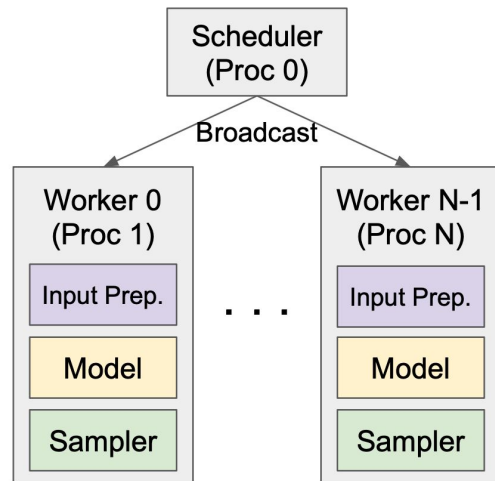
Особенности:

- Постоянное добавление/удаление запросов
- Выравнивание по длинам

V0 Architecture



V1 Architecture



Quantization & Hardware

- В зависимости от железа оптимизации
- Например, H200 поддерживает FP8
- FlashAttention
- ...

Implementation	Volta	Turing	Ampere	Ada	Hopper	AMD GPU	Intel GPU	Intel Gaudi	x86 CPU
AWQ	✗	✓	✓	✓	✓	✗	✓	✗	✓
GPTQ	✓	✓	✓	✓	✓	✗	✓	✗	✓
Marlin (GPTQ/AWQ/FP8)	✗	✗	✓	✓	✓	✗	✗	✗	✗
INT8 (W8A8)	✗	✓	✓	✓	✓	✗	✗	✗	✓
FP8 (W8A8)	✗	✗	✗	✓	✓	✓	✗	✗	✗
BitBLAS	✓	✓	✓	✓	✓	✗	✗	✗	✗
BitBLAS (GPTQ)	✗	✗	✓	✓	✓	✗	✗	✗	✗
bitsandbytes	✓	✓	✓	✓	✓	✗	✗	✗	✗
DeepSpeedFP	✓	✓	✓	✓	✓	✗	✗	✗	✗
GGUF	✓	✓	✓	✓	✓	✓	✗	✗	✗
INC (W8A8)	✗	✗	✗	✗	✗	✗	✗	✓	✗

- Observability (мониторинг)
- API
- Guided output
- Memory management
- ...

vLLM

- Paged Attention
- LLM, encoders, cross-encoders
- Structured output

Paged Attention

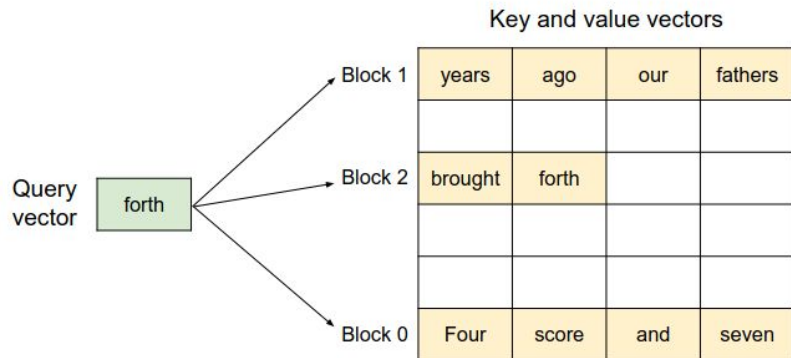


Figure 5. Illustration of the PagedAttention algorithm, where the attention key and values vectors are stored as non-contiguous blocks in the memory.

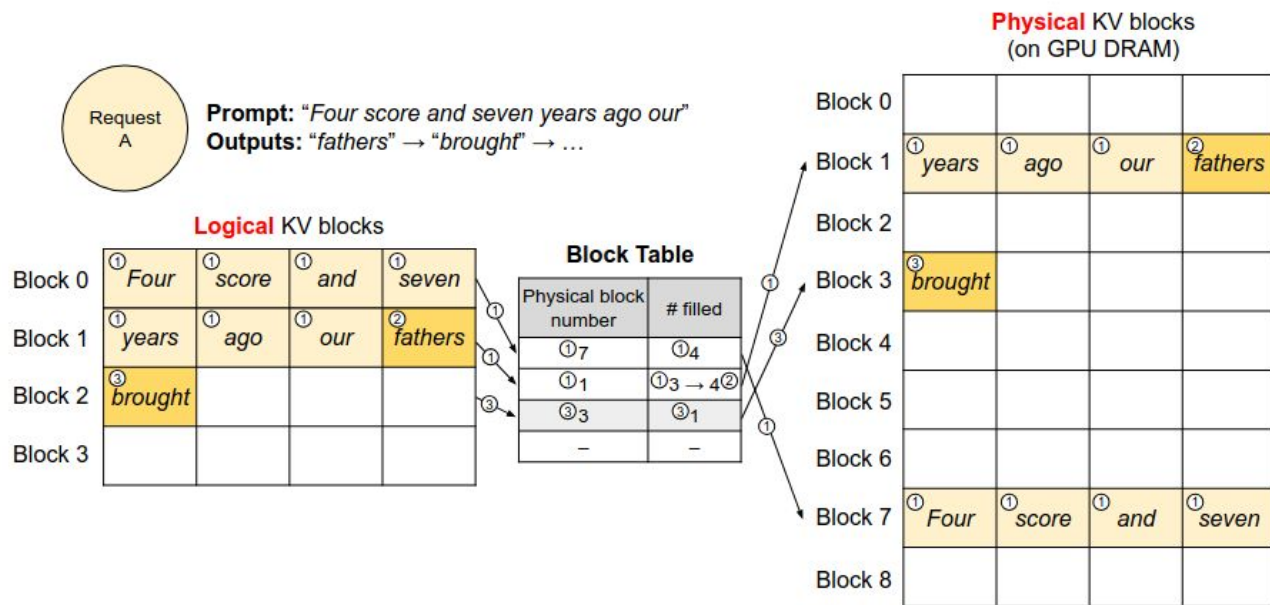


Figure 6. Block table translation in vLLM.

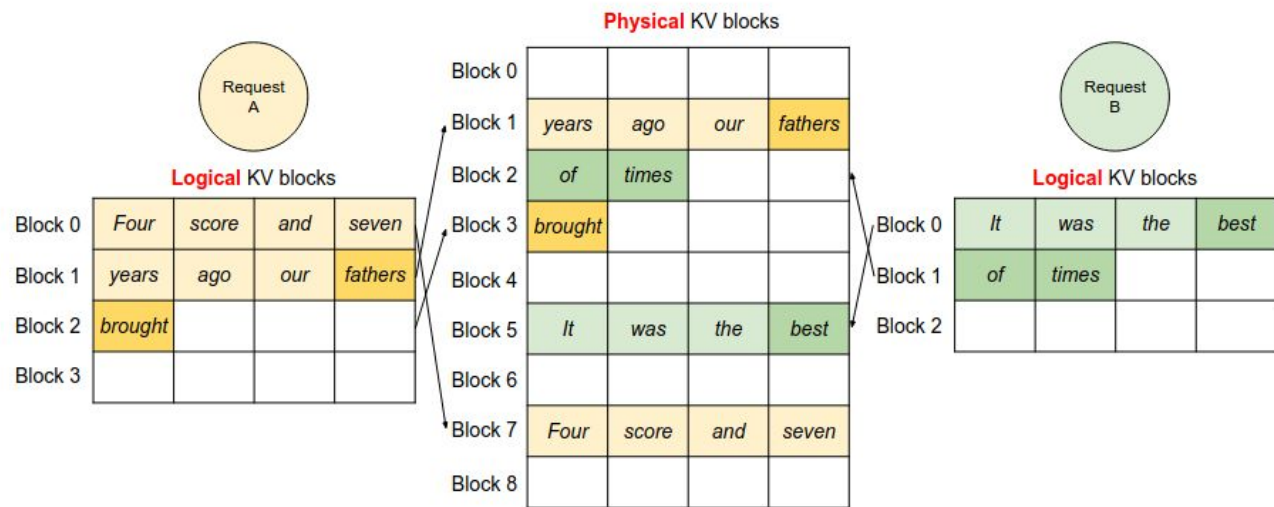


Figure 7. Storing the KV cache of two requests at the same time in vLLM.

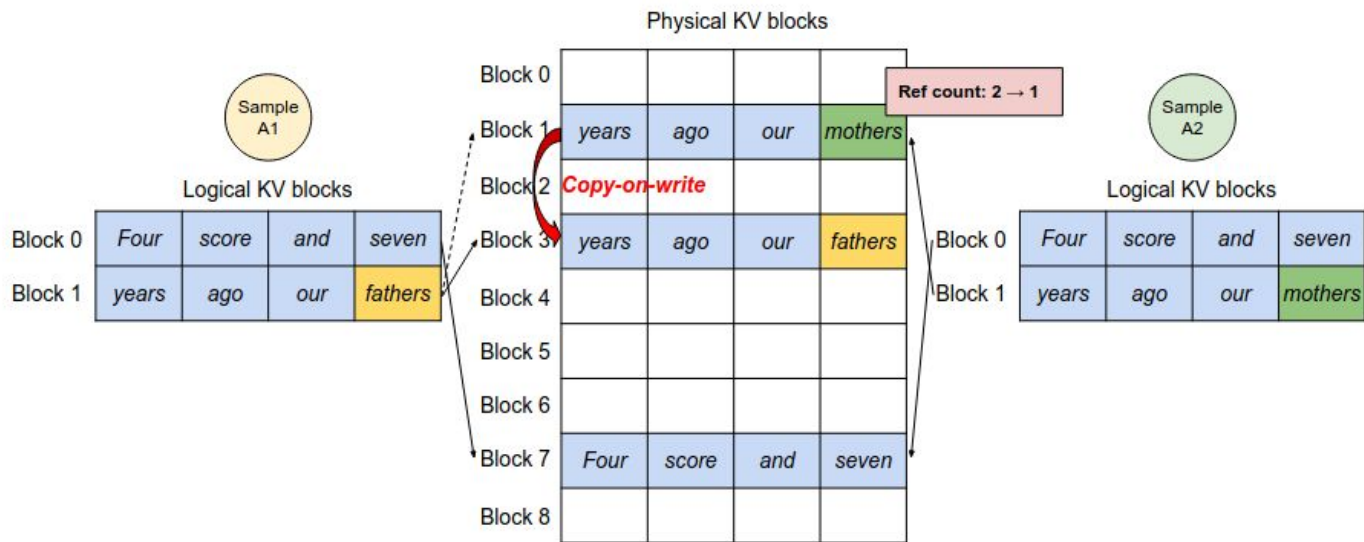
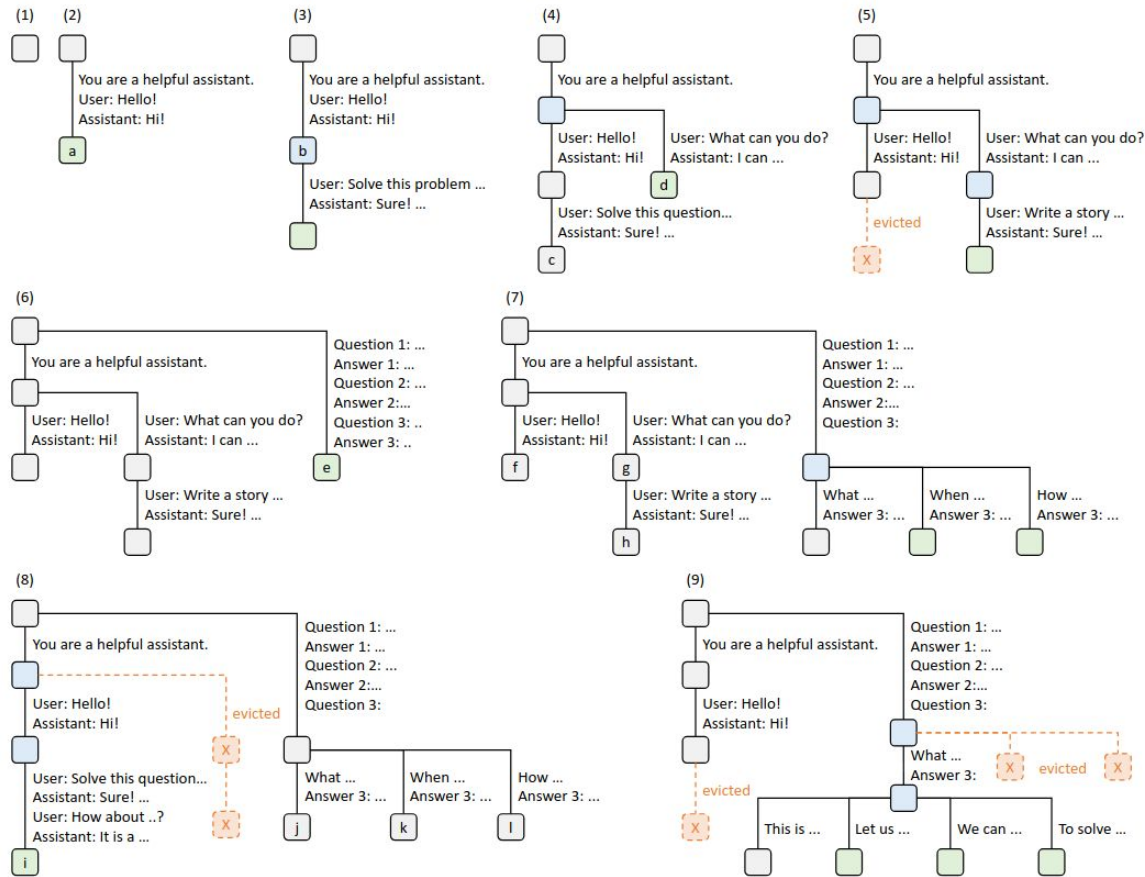


Figure 8. Parallel sampling example.

SGLang

- Radix Attention
- LLM, encoders

RadixAttention



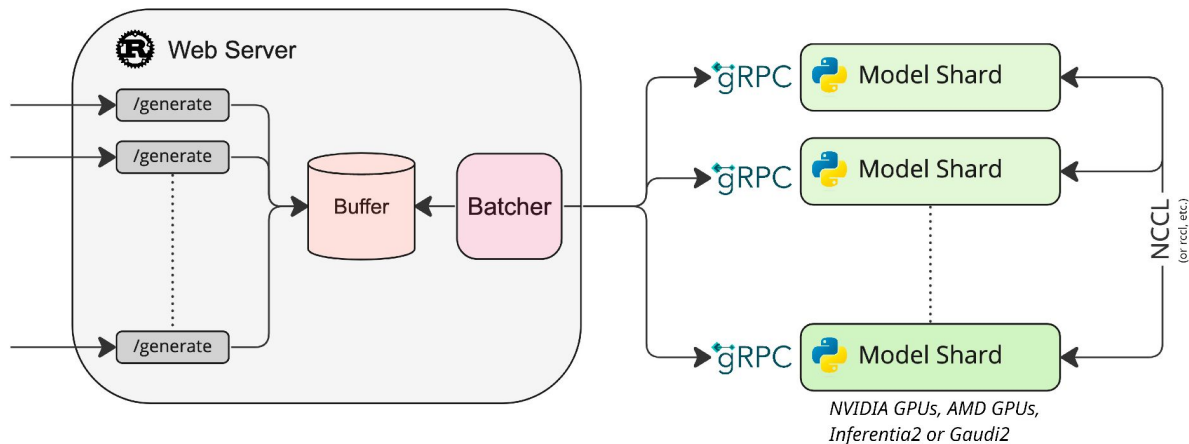
TGI

- Paged Attention
- LLM
- Multiple backends

(TensorRT, llama.cpp)

Text Generation Inference

Fast optimized inference for LLMs



Спасибо за внимание